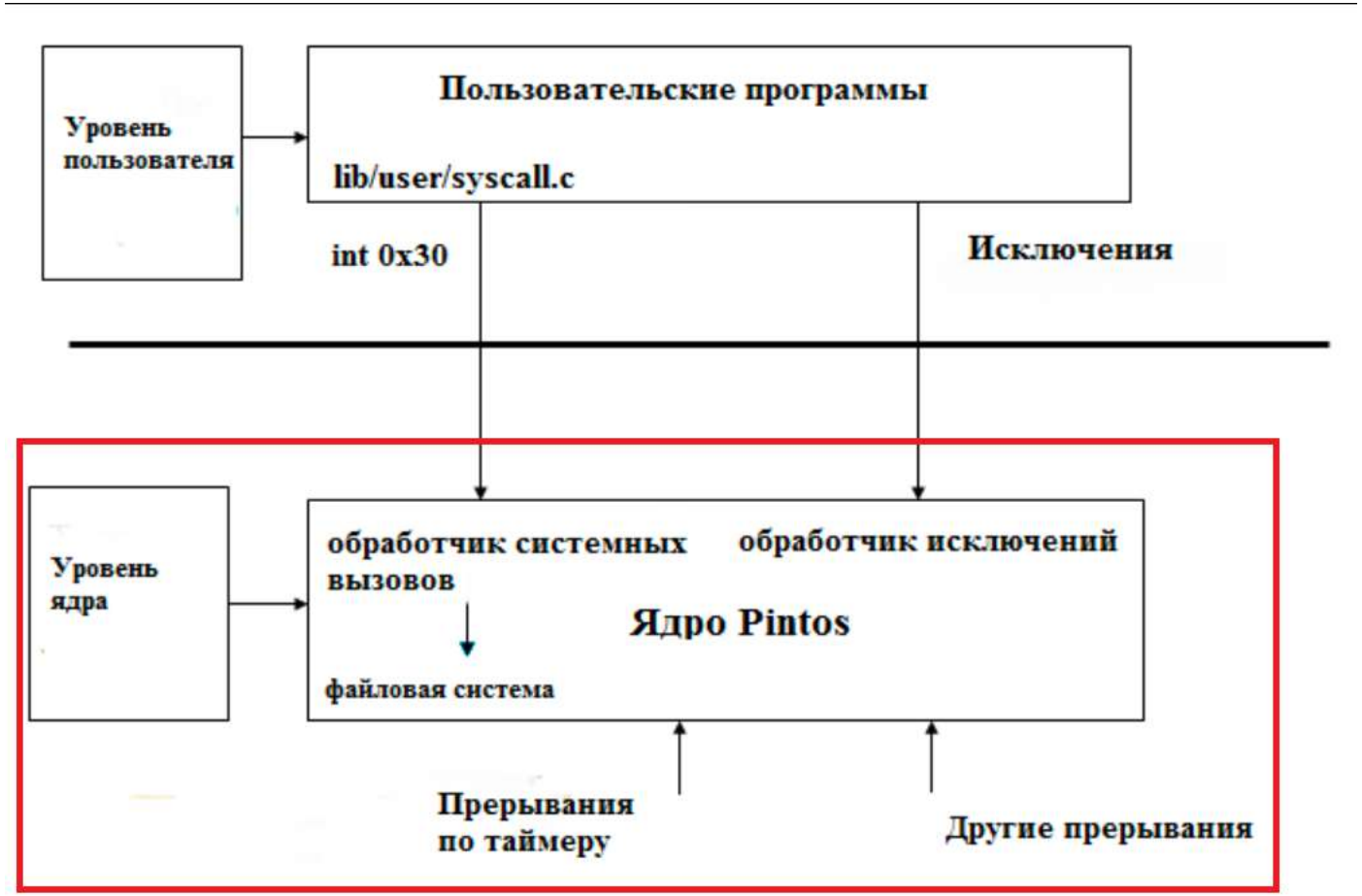
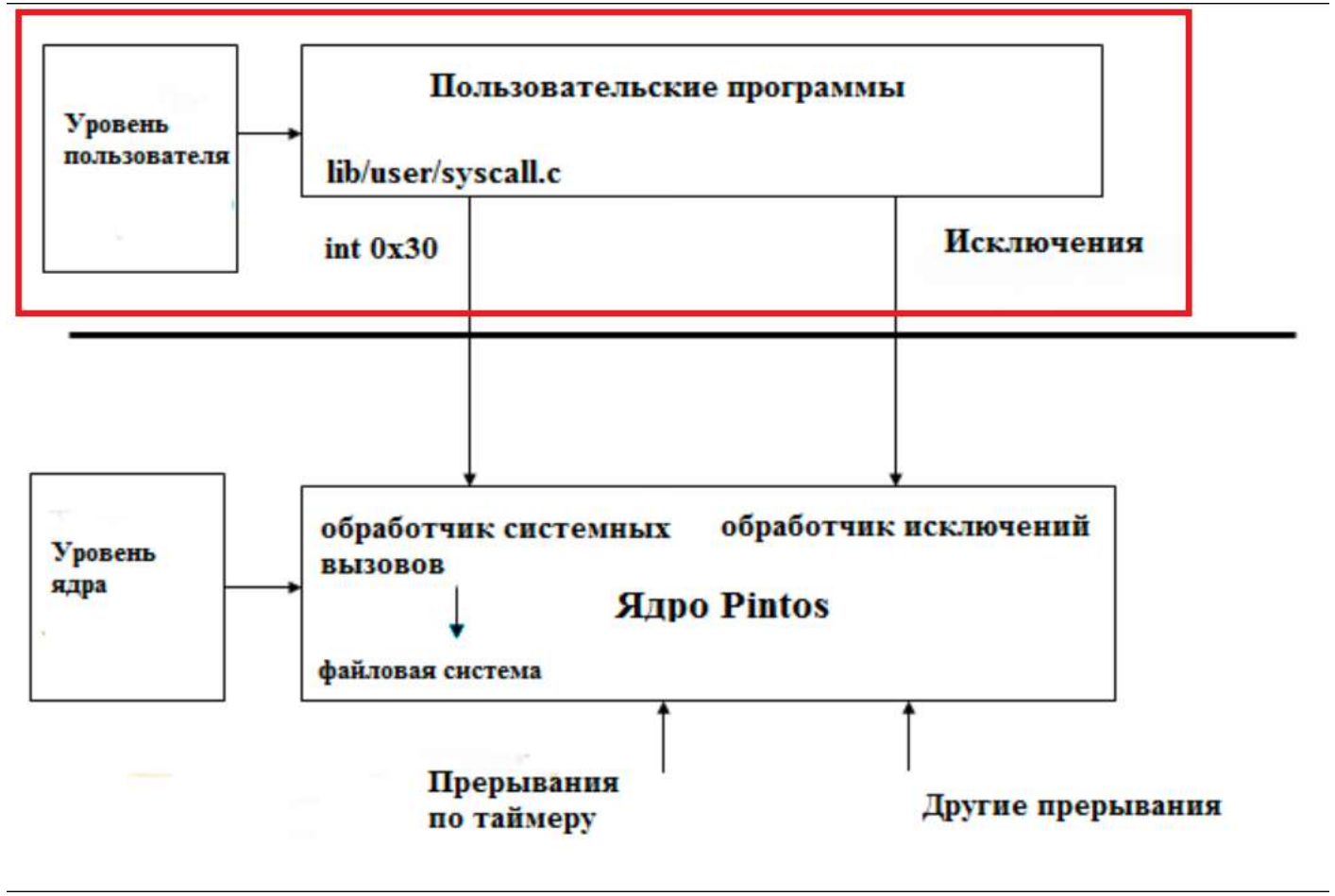
Пятая работа

Уровни взаимодействия в ОС Pintos



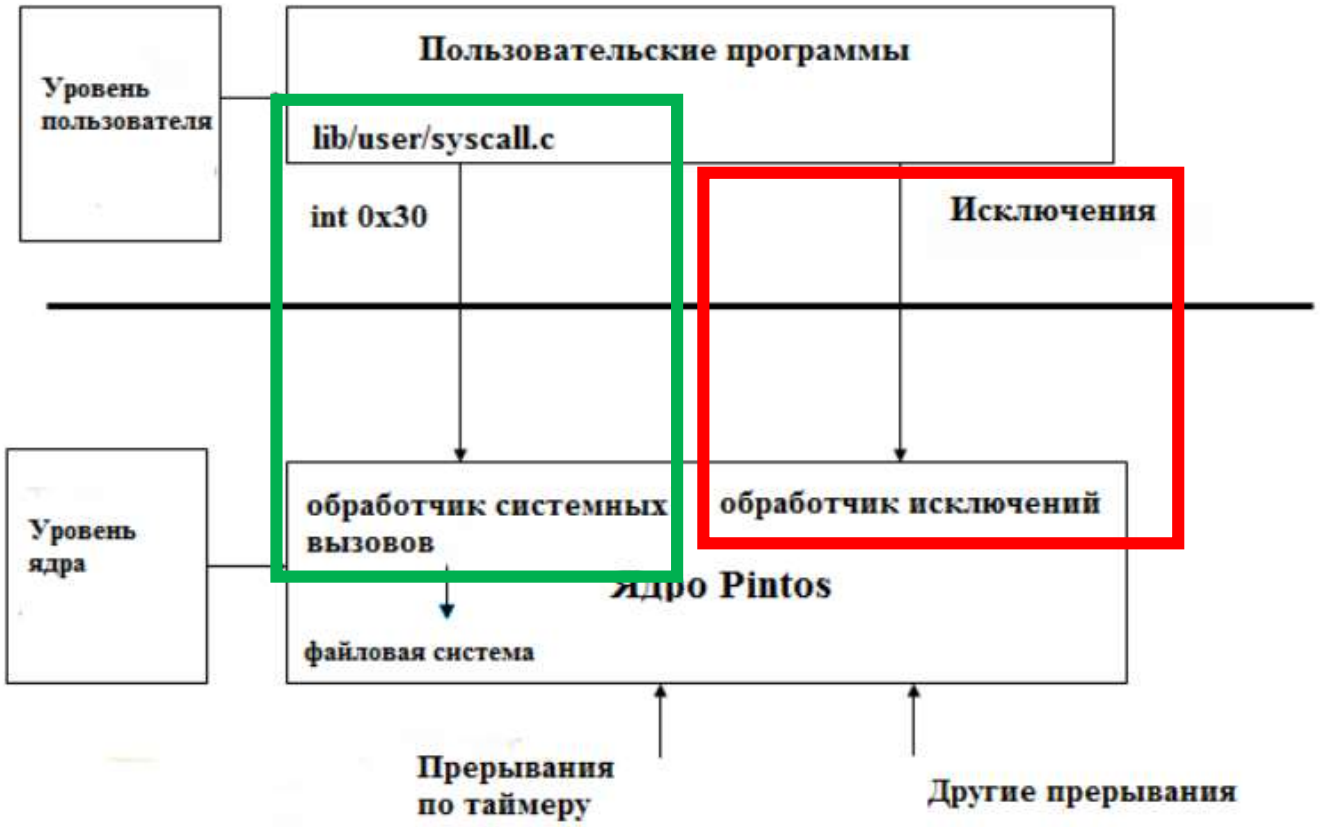
Работали на этом уровне на протяжении 1-3 лабораторных работ

Уровни взаимодействия в ОС Pintos



На протяжении четвертой работы работы с пользовательским уровнем

Уровни взаимодействия в ОС Pintos



В рамках пятой работы ведется работа над реализацией механизмов прерываний и системных вызовов

Системные вызовы и прерывания — механизмы, позволяющие управлять взаимодействием между пользовательскими программами и аппаратным обеспечением компьютера

Системные вызовы (System Calls)

Это программный интерфейс, через который пользовательские программы **запрашивают «услуги» у операционной системы**

Примеры: создание и управление процессами (*fork, exec*), управление файлами (*open, read, write, close*), обработка сетевых запросов и взаимодействие с устройствами

Базовая реализация системных вызовов в ОС Pintos размещает аргументы с соответствующим номером системного вызова в регистры и вызывает прерывание 0x30

Это прерывание заставляет процессор остановить выполнение текущего задания, переключиться из режима пользователя в режим ядра и перейти к обработчику прерываний syscall_handler, который определен в userprog/syscall.c

Прерывания (Interrupts)

Это механизм, с помощью которого аппаратное обеспечение может сигнализировать о необходимости немедленного внимания операционной системы

Примеры: обработка асинхронных событий, таких как ввод данных с клавиатуры, прием пакетов сетевого интерфейса или сигналы таймера

	Источник	Пример	Вид влияния	Могут быть отключены
Внутрение	Процессор	Деление на ноль, обращение по недопустимым адресам	Синхронные	Нет
Внешние	Внешние устройства	Прерывания таймера, клавиатуры	Асинхронные	Да

Инициализация прерываний в ОС Pintos

В функции intr_init() происходит установка таблицы векторов прерываний и связывание каждой точки входа с функцией intrNN_stub()

```c
/* Initializes the interrupt system. */
void intr_init (void)
{
  uint64_t idtr_operand;
  int i;

  /* Initialize interrupt controller. */
  pic_init ();

  /* Initialize IDT. */
  for (i = 0; i < INTR_CNT; i++)
    idt[i] = make_intr_gate (intr_stubs[i], 0);

  /* ... */

  /* Initialize intr_names. */
  for (i = 0; i < INTR_CNT; i++)
    intr_names[i] = "unknown";
  intr_names[0] = "#DE Divide Error";
  intr_names[1] = "#DB Debug Exception";
  intr_names[2] = "NMI Interrupt";
  intr_names[3] = "#BP Breakpoint Exception";
  intr_names[4] = "#OF Overflow Exception";
  /* ... */
  intr_names[18] = "#MC Machine-Check Exception";
  intr_names[19] = "#XF SIMD Floating-Point Exception";
}
```

Названия обработчиков

```c
/* Interrupt stubs.

   This defines 256 fragments of code, named `intr00_stub'
   through `intrff_stub', each of which is used as the entry
   point for the corresponding interrupt vector.  It also puts
   the address of each of these functions in the correct spot in
   `intr_stubs', an array of function pointers.

   Most of the stubs do this:

        1. Push %ebp on the stack (frame_pointer in `struct intr_frame').

        2. Push 0 on the stack (error_code).

        3. Push the interrupt number on the stack (vec_no).

   The CPU pushes an extra "error code" on the stack for a few
   interrupts.  Because we want %ebp to be where the error code
   is, we follow a different path:

        1. Push a duplicate copy of the error code on the stack.

        2. Replace the original copy of the error code by %ebp.

        3. Push the interrupt number on the stack. */
```

```c
/* All the stubs. */
STUB(00, zero) STUB(01, zero) STUB(02, zero) STUB(03, zero)
STUB(04, zero) STUB(05, zero) STUB(06, zero) STUB(07, zero)
STUB(08, REAL) STUB(09, zero) STUB(0a, REAL) STUB(0b, REAL)
STUB(0c, zero) STUB(0d, REAL) STUB(0e, REAL) STUB(0f, zero)

/* ... */

STUB(f0, zero) STUB(f1, zero) STUB(f2, zero) STUB(f3, zero)
STUB(f4, zero) STUB(f5, zero) STUB(f6, zero) STUB(f7, zero)
STUB(f8, zero) STUB(f9, zero) STUB(fa, zero) STUB(fb, zero)
STUB(fc, zero) STUB(fd, zero) STUB(fe, zero) STUB(ff, zero)
```

Обработка прерываний в ОС Pintos

Основная работа функции intr_handler() заключается в вызове функции, предназначенной для обработки конкретного прерывания (handler)

Если такой функции не существует, intr_handler() завершается ошибкой и выводит информацию на консоль

Когда intr_handler() завершает работу, ассемблерные функции в threads/intr-stubs.S восстанавливают ранее сохраненное состояние всех регистров процессора и передают управление процессору

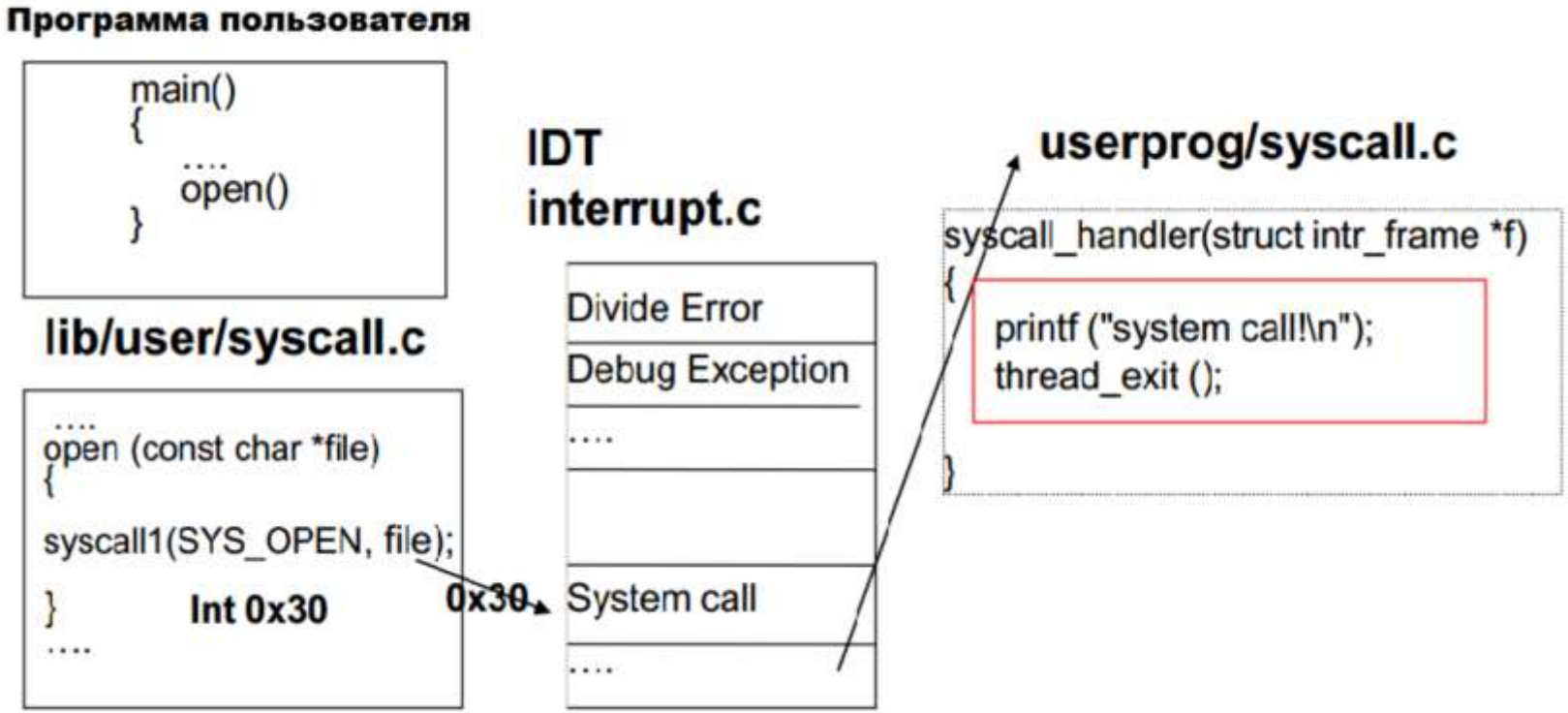
```
/* Handler for all interrupts, faults, and exceptions.  This
   function is called by the assembly language interrupt stubs in
   intr-stubs.S.  FRAME describes the interrupt and the
   interrupted thread's registers. */
void intr_handler (struct intr_frame *frame)
{
  /* External interrupts are special.
     We only handle one at a time (so interrupts must be off)
     and they need to be acknowledged on the PIC (see below).
     An external interrupt handler cannot sleep. */
  external = frame->vec_no >= 0x20 && frame->vec_no < 0x30;
  if (external)
    {
      in_external_intr = true;
      yield_on_return = false;
    }

  /* Invoke the interrupt's handler. */
  handler = intr_handlers[frame->vec_no];
  if (handler != NULL)
    handler (frame);
  else
    unexpected_interrupt (frame);

  /* Complete the processing of an external interrupt. */
  if (external)
    {
      in_external_intr = false;
      pic_end_of_interrupt (frame->vec_no);

      if (yield_on_return)
        thread_yield ();
    }
}
```

Базовая обработка системных вызовов в ОС Pintos



Обработка системных вызовов в ОС Pintos

```
void
syscall_init (void)
{
  intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");
}

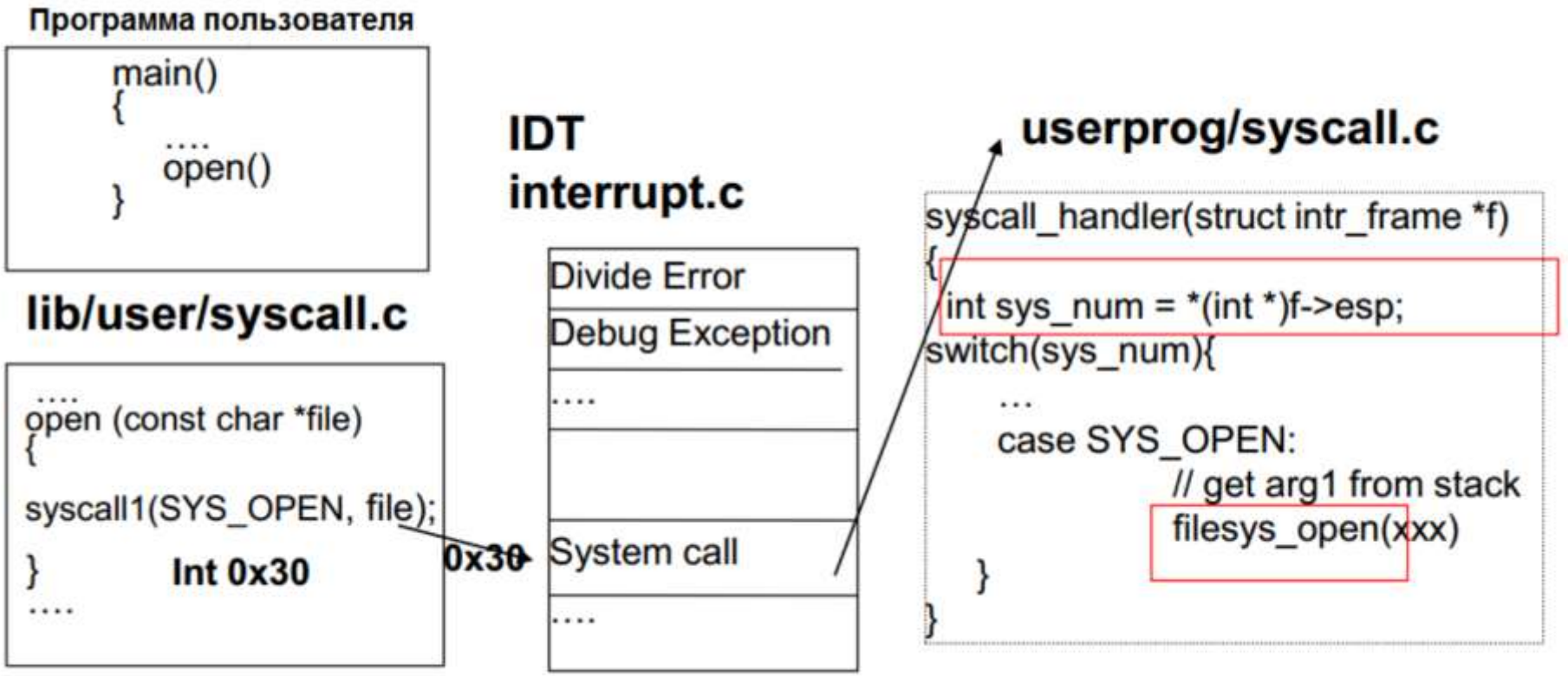
static void
syscall_handler (struct intr_frame *f UNUSED)
{
  printf ("system call!\n");
  thread_exit ();
}
```

Реализация обработчика системных вызовов по умолчанию в ОС Pintos

```
/* System call numbers. */
enum
  {
    SYS_HALT,                   /* Halt the operating system. */
    SYS_EXIT,                   /* Terminate this process. */
    SYS_EXEC,                   /* Start another process. */
    SYS_WAIT,                   /* Wait for a child process to die. */
    SYS_CREATE,                 /* Create a file. */
    SYS_REMOVE,                 /* Delete a file. */
    SYS_OPEN,                   /* Open a file. */
    SYS_FILESIZE,               /* Obtain a file's size. */
    SYS_READ,                   /* Read from a file. */
    SYS_WRITE,                  /* Write to a file. */
    SYS_CLOSE,                  /* Close a file. */
  };
```

Системные вызовы, поддержку которых необходимо реализовать по заданию

Модифицированная версия обработчика системных вызовов



Задачи

1. Модифицировать текущий механизм обработки системных вызовов (добавить чтение и валидацию аргументов, чтение номера системного вызова, реализация корректного завершения обработки с возвращением значения)

2. Расширить перечень поддерживаемых системных вызовов